

SOC 2 Logging Architecture (Admin + Global Admin)

1. Objective

Build a SOC 2-aligned logging architecture for this platform that:

- Provides complete and reliable evidence for privileged actions.
- Separates operational logs from compliance audit logs.
- Enforces clear behavior differences for `admin` and `global\_admin`.
- Supports investigations, incident response, and external audits.

This document is implementation-ready and tailored to the current stack:

- Frontend: Next.js + Supabase client
- Backend: FastAPI
- Database/Auth: Supabase (Postgres + RLS)

2. Core Principles

1. **Dual logging model**
 - **Application logs** for reliability and debugging.
 - **Audit logs** for compliance and legal evidence.
2. **Backend as source of truth**
 - Privileged actions must be executed and logged by backend APIs.
 - Frontend must not directly perform privileged writes.
3. **Least privilege**
 - `admin` handles tenant/domain administration.
 - `global\_admin` handles cross-tenant and high-risk controls.
4. **Tamper resistance**
 - Audit events are append-only for normal operators.
5. **Data minimization**
 - Log metadata and identifiers, not secrets or full sensitive payloads.

3. Architecture Overview

3.1 Layers

1. **Request Context Layer (FastAPI middleware)**
 - Generates `request\_id`.
 - Captures `actor\_user\_id`, `actor\_role`, `ip\_address`, `user\_agent`.

- Adds timing metrics (`duration\_ms`).

2. **Operational Logger**

- Structured JSON logs to stdout or log sink.
- Fields: `timestamp`, `level`, `service`, `route`, `request\_id`, `status\_code`, `duration\_ms`, `error\_code`.
- Use for uptime, debugging, and performance.

3. **Audit Service (centralized)**

- Shared backend service to emit audit events for security-sensitive actions.
- Called from all privileged endpoints.
- Writes to `audit\_logs` (or `audit\_events`) with strict schema.

4. **Audit Storage + Access Control**

- Supabase Postgres table with RLS:
- `admin`: read scoped events.
- `global\_admin`: read global events.
- No updates/deletes for normal roles.

5. **Detection and Alerting**

- Alert rules for:
- Role changes
- Failed privileged actions
- Repeated authorization denials
- Break-glass usage

6. **Evidence and Retention**

- Defined retention windows (hot + archive).
- Scheduled export for SOC 2 evidence packets.

4. Role-Centric Control Model

4.1 `admin` role

- Can execute tenant/domain operations (for example: approve pod activation, allocate models).
- Can read audit events within allowed scope.
- Cannot:
- Elevate users to `global\_admin`
- Change logging policy
- Disable controls or retention settings

4.2 `global\_admin` role

- Can execute high-impact global operations:
 - Role elevation/demotion
 - Cross-tenant user access changes
 - Security policy and control changes
- Requires stronger evidence fields:
 - `justification`
 - `ticket\_id`
 - optional `approver\_id` / change reference

4.3 Control pattern

- Every privileged API must:
 1. Authenticate actor.
 2. Resolve role.
 3. Authorize action.
 4. Emit audit event (`attempt` and `result`).

5. Audit Event Taxonomy

Use stable event names with versioning (`v1`) to keep analytics and audits consistent.

5.1 Recommended events

- `identity.user.created.v1`
- `identity.user.password\_updated.v1`
- `identity.user.role\_updated.v1`
- `access.denied.v1`
- `device.pairing.started.v1`
- `device.pairing.confirmed.v1`
- `device.registration.updated.v1`
- `pod.request.approved.v1`
- `pod.request.rejected.v1`
- `model.allocation.created.v1`
- `model.allocation.removed.v1`
- `security.break\_glass.started.v1`
- `security.break\_glass.ended.v1`

5.2 Event severity

- `info`: expected admin operations
- `warning`: policy denials / unusual patterns
- `critical`: global role elevation, break-glass, control disablement attempts

6. Audit Event Schema (Recommended)

```
{
  "event_id": "uuid",
  "timestamp": "2026-03-13T12:45:10.130Z",
  "event_name": "identity.user.role_updated.v1",
  "result": "success",
  "severity": "critical",
  "request_id": "req_9f9f1f4b",
  "actor": {
    "user_id": "e8f9...123",
    "role": "global_admin"
  },
  "target": {
    "resource_type": "user_role",
    "resource_id": "6a6f...abc",
    "tenant_id": "tenant_default"
  },
  "change": {
    "before": {"role": "admin"},
    "after": {"role": "global_admin"}
  },
  "context": {
    "ip_address": "203.0.113.10",
    "user_agent": "Mozilla/5.0 ...",
    "route": "/api/admin-set-user-role",
    "method": "POST"
  },
  "compliance": {
    "justification": "On-call leadership rotation",
    "ticket_id": "SEC-4821"
  }
}
```

7. Data Classification and Redaction

Never log

- Access tokens, refresh tokens, service keys, API keys
- Raw passwords
- Full device secret tokens

Log with masking/hashing

- User email -> partial mask if needed
- Device token -> hash + short fingerprint
- Prompt content -> only if approved by policy; otherwise metadata only

Safe fields

- IDs, roles, route/method, status code, action type, timestamps

8. Demo Examples

8.1 Example A: Admin approves pod request

****Action:**** `admin` approves a pending pod activation request.

****Audit event (success):****

```
{
  "event_name": "pod.request.approved.v1",
  "result": "success",
  "severity": "info",
  "actor": {"user_id": "u_admin_123", "role": "admin"},
  "target": {"resource_type": "pod_activation_request", "resource_id": "req_456"},
  "change": {"before": {"status": "pending"}, "after": {"status": "approved"}}
}
```

8.2 Example B: Global admin changes user role

****Action:**** `global\_admin` updates user role from `user` to `admin`.

****Audit event (success + elevated controls):****

```
{
  "event_name": "identity.user.role_updated.v1",
  "result": "success",
  "severity": "critical",
  "actor": {"user_id": "u_global_001", "role": "global_admin"},
  "target": {"resource_type": "user_role", "resource_id": "u_target_999"},
  "change": {"before": {"role": "user"}, "after": {"role": "admin"}},
  "compliance": {"justification": "Team lead onboarding", "ticket_id": "IAM-2203"}
}
```

8.3 Example C: Unauthorized role-change attempt

****Action:**** normal user attempts role update endpoint.

****Audit event (denied):****

```
{
  "event_name": "access.denied.v1",
  "result": "denied",
  "severity": "warning",
  "actor": {"user_id": "u_user_777", "role": "user"},
  "target": {"resource_type": "endpoint", "resource_id": "/api/admin-set-user-role"},
  "context": {"reason": "Global admin access required"}
}
```

9. API Coverage Matrix (Current Project Focus)

Prioritize these backend routes first:

1. ``/api/admin-create-user``
 - Events: ``identity.user.created.v1``, ``identity.user.password_updated.v1``
2. ``/api/admin-set-user-role``
 - Events: ``identity.user.role_updated.v1`` (critical)
3. ``/api/list-users``
 - Event: ``identity.user.listed.v1`` (optional info event)
4. ``/api/pod-activation/{request_id}/approve``
 - Event: ``pod.request.approved.v1``
5. ``/api/pod-activation/{request_id}/reject``
 - Event: ``pod.request.rejected.v1``
6. ``/api/pod-activation/allocate``
 - Event: ``model.allocation.created.v1``
7. ``/api/pod-activation/allocations/{allocation_id}`` (delete)
 - Event: ``model.allocation.removed.v1``
8. ``/api/devices/pairing/start`` and ``/api/devices/pairing/confirm``
 - Events: ``device.pairing.started.v1``, ``device.pairing.confirmed.v1``

10. SOC 2 Evidence Strategy

Generate monthly evidence bundles containing:

- Role change log (who changed what, when, why)
- Denied privileged attempts
- Global admin actions with ticket references
- Alert incidents and response notes
- Retention and integrity verification report

Recommended cadence:

- Daily: integrity checks and alert review
- Weekly: access anomalies review
- Monthly: formal evidence export

11. Implementation Phases

Phase 1: Foundation (Week 1)

- Introduce request context middleware.
- Introduce centralized audit service interface.
- Standardize JSON operational logging format.

Phase 2: Privileged route coverage (Week 1-2)

- Instrument all admin/global admin endpoints.
- Add denied-attempt logging.
- Enforce required fields for critical events.

Phase 3: Detection + retention (Week 2)

- Configure SIEM dashboards and alerts.
- Configure retention and archive policy.
- Add monthly evidence export job.

Phase 4: Validation and audit-readiness (Week 3)

- Run test scenarios and verify every event chain.
- Review false positives in alerting.
- Sign off with security/compliance stakeholders.

12. Verification Checklist

- ☐ Every privileged endpoint emits an audit event.
- ☐ Denied access attempts are logged with actor + reason.
- ☐ Role changes include before/after state and ticket reference.
- ☐ Audit events are append-only for non-system roles.
- ☐ Sensitive values are redacted/masked.
- ☐ Alert rules fire for critical events.
- ☐ Evidence export is repeatable and timestamped.

13. Quick Demo Payload Set

Use these payloads for dry-run verification.

13.1 Promote user to admin

```
{"user_id":"3d5f...", "role":"admin", "justification":"Ops support expansion", "ticket_id":"IAM-145"}
```

13.2 Reject pod request

```
{"rejection_reason":"Domain mismatch with current policy"}
```

13.3 Create user

```
{"email":"new.user@example.com", "password":"TempPass#1234", "full_name":"New User", "role":"viewer"}
```

Expected log outcomes:

- One operational log per request lifecycle.
- One compliance audit event per sensitive action.
- Warning event for every denied request.

14. Final Recommendation

The best fit for this project is:

- Keep **Supabase audit storage** and RLS controls.
- Add a **single backend audit service** for all sensitive writes.
- Enforce strict role-based control differences between ``admin`` and ``global_admin``.
- Treat logs as a formal security control and evidence pipeline, not just debugging output.

This gives strong SOC 2 alignment with minimal architectural disruption.